



Gym Membership Management System

In this lab, imagine **John**, a fitness enthusiast, who recently joined **FitLife Gym** managed by **Sarah**, the gym's admin. **Sarah** is responsible for overseeing the overall operations, including managing the trainers and ensuring a seamless experience for the members. One of the gym's senior trainers, **David**, is assigned to help John reach his fitness goals. David specialises in strength training and yoga and manages several classes for members like John.

The gym operates on a structured membership system. Members like John can sign up for different membership tiers (e.g., monthly, yearly, or premium). Once registered, they can view and join the available classes, taught by the trainers. David's classes have limited seats, so members need to check availability and register early. If for any reason a member wants to cancel or reschedule, the system allows for easy modifications.

On the back end, **Sarah** adds and removes trainers from the system as necessary, ensuring that every trainer's information is up to date. Trainers like David are responsible for managing member registrations for classes, checking attendance, and updating their availability. They can also add new classes, track the remaining seats, and manage member requests for cancellations.

The System Classes:

The **Admin** is responsible for adding new trainers to the system and removing the data of the trainers who no longer work for the gym.

To model the admin and his responsibilities in our system, we need to define a few classes.

First Class:

we need to define a class named **Trainer** to represent a trainer's personal information. There are five private String variables in the class: **trainerId**, **name**, **email**, **speciality**, and **phoneNumber**.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. AbdElrahman ElSayed



No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single constructor that initializes the trainer instance attributes.

The class contains two methods:

1. **lineRepresentation ()**

Description: This method returns the trainer's data in a comma-separated format (e.g., T1001, John, john@gmail.com, Yoga,0123456789). This format will be used for saving trainer information in a file.

2. **getSearchKey ()**

Description: This method returns the trainer's unique ID (trainerId). It will be used as a search key when accessing trainer records from the system.

The data of the trainers is stored in a file named 'Trainers.txt'. Each line represents a data record of one trainer

Second Class:

Define a class named **TrainerDatabase** which is responsible for accessing the file of the trainers (reading from and writing to), accessing, and manipulating (adding to and removing from) the list of **Trainer** objects that hold the data read from the file. There are two private variables in the class: **records (ArrayList<Trainer>)**, and **filename (String)**.

No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single constructor, which takes only the file name as parameter Then initialize records ArrayList to prepare the system to handle trainer data by reading from and writing to the file.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. AbdElrahman ElSayed



The class contains eight methods:

1. **readFromFile ()**
Description: This method reads all trainer records from the file Trainers.txt, converts each line into a Trainer object, and stores it in the records list.
2. **createRecordFrom (String line)**
Description: This method takes a comma-separated string (line) as input and returns a Trainer object created from the data in the string.
3. **returnAllRecords ()**
Description: This method returns the entire list of trainer records stored in the records array list. It will be used to retrieve all trainers in the system.
4. **contains (String key)**
Description: This method checks if a trainer with the given key (trainer ID) exists in the records list. It returns true if the trainer is found, otherwise false.
5. **getRecord (String key)**
Description: This method searches for a trainer by their trainerId (key). It returns the Trainer object if the trainer is found, otherwise returns null.
6. **insertRecord (Trainer record)**
Description: This method inserts a new Trainer object into the records list. It also checks if the trainer already exists (based on their trainerId), preventing duplicate entries.
7. **deleteRecord (String key)**
Description: This method removes a Trainer from the records list based on their trainerId.
8. **saveToFile ()**
Description: This method writes all the current trainer records in the records list back to Trainers.txt. It overwrites the file with the most up-to-date information.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed	Eng. Ali Hassan
Eng. Ahmed Ashraf	Eng. Seif Eldin Mahmoud
Eng. Miar Mamdouh	Eng. Ahmed AboEleid
Eng. Ahmed Essam	Eng. Menna Tullah Ihab
Eng. Abdelaziz Mohamed	Eng. Mahmoud Ramzy
Eng. Abdelrahman ElSayed	



Third Class:

Define a class named **AdminRole** that represents the role of the admin. The class contains only one private variable named **database** which is an object from the **TrainerDatabase** class.

No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single default constructor, which doesn't take any value as parameters to prepare the system to manage trainers by accessing the TrainerDatabase.

The class contains four methods:

1. **addTrainer (String trainerId, String name, String email, String specialty, String phoneNumber)**
Description: This method adds a new trainer to the system. It creates a new Trainer object and adds it to the TrainerDatabase. It checks for duplicates before adding a new trainer.
2. **getListOfTrainers ()**
Description: This method retrieves a list of all the trainers in the system. It returns an array containing all Trainer objects stored in the TrainerDatabase.
3. **removeTrainer (String key)**
Description: This method removes a trainer from the system by their trainerId.
4. **logout ()**
Description: This method ensures all unsaved trainer data is written back to Trainers.txt before the admin logs out.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed	Eng. Ali Hassan
Eng. Ahmed Ashraf	Eng. Seif Eldin Mahmoud
Eng. Miar Mamdouh	Eng. Ahmed AboEleid
Eng. Ahmed Essam	Eng. Menna Tullah Ihab
Eng. Abdelaziz Mohamed	Eng. Mahmoud Ramzy
Eng. Abdelrahman ElSayed	



Now, we will describe the role of the trainer, and the classes needed for this role.

The trainer is responsible for adding new members to the system, adding new classes, registering members in classes, and cancelling them if members want to. So, we need some more classes to apply for the trainer's role.

First Class:

we need to define a class named **Member**. This class represents a gym member's information and includes fields for storing membership details. There are six private String variables in the class: **memberID, name, membershipType, email, phoneNumber, and status** (active, expired, etc.).

No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single constructor that initializes the member instance attributes.

The class contains two methods:

1. **lineRepresentation ()**

Description: This method returns the member's data in a comma-separated format, used for saving member information in a file (e.g., M1001, John, Monthly, john@gmail.com,01012345678, active).

2. **getSearchKey ()**

Description: This method returns the member's ID (memberID). It will be used to search for a member in the system.

The data of the members is stored in a file named '**Members.txt**'. Each line represents a data record of one member.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. AbdElrahman ElSayed



Second Class:

Define a class named **MemberDatabase** which is responsible for accessing the file of the members (reading from and writing to), accessing, and manipulating (adding to and removing from) the list of **Member** objects that hold the data read from the file. There are two private variables in the class: **records (ArrayList<Member>)**, and **filename (String)**.

No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single constructor, which takes only the file name as Then initialize records ArrayList to prepare the system to handle members' data.

The class contains eight methods:

1. **readFromFile ()**
Description: Reads all member records from Members.txt, converts each line into a member object, and stores it in the records list.
2. **createRecordFrom (String line)**
Description: Takes a comma-separated string (line) and returns a member object created from the data in the string.
3. **returnAllRecords ()**
Description: Returns all member records stored in the records list.
4. **contains (String key)**
Description: Checks if a member with the given memberID (key) exists in the records list.
5. **getRecord (String key)**
Description: Returns the Member object corresponding to the provided memberID, or null if not found.
6. **insertRecord (Member record)**
Description: Adds a new Member to the records list if the member doesn't already exist.
7. **deleteRecord (String key)**
Description: Removes the member with the specified memberID from the records list and updates the file.
8. **saveToFile ()**
Description: Writes all the member records stored in the records list back to the file.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. Abdelrahman ElSayed



Third Class:

we need to define a class named **Class**. This class represents gym class information and includes fields for storing its details. There are three private String variables in the class: **classID**, **className**, and **trainerID (trainer who will give session)**, There are two private Int variables in the class: **duration**, and **availableSeats**.

For Simplicity: We will assume that every class has a single appointment, so everyone registered in the class registers the same appointment.

No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single constructor that initializes the class instance attributes.

The class contains four methods:

1. **getAvailableSeats ()**
Description: Returns the number of available seats in the current class.
2. **setAvailableSeats (int availableSeats)**
Description changes the number of available seats in the current class.
3. **lineRepresentation ()**
Description: Returns the class information in a comma-separated format (e.g., C101, Yoga, T001,60,30), used for saving class information in a file.
4. **getSearchKey ()**
Description: Returns the class's ID (classID). It will be used as the search key in the class management system.

The data of the classes is stored in a file named '**Class.txt**'. Each line represents a data record of one class.

Fourth Class:

Define a class named **ClassDatabase** which is responsible for accessing the file of the **Class.txt** (reading from and writing to), accessing, and manipulating (adding to and removing from) the list of **Class** objects that hold the data read from the file. There are two private variables in the class: **records (ArrayList<Class>)**, and **filename (String)**.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. Abdelrahman ElSayed



No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single constructor, which takes only the file name as parameter Then initializes records ArrayList to prepare the system to handle class data by reading and writing to this file.

The class contains eight methods:

1. **readFromFile ()**
Description: Reads all class records from Classes.txt, converts them into Class objects, and stores them in the records list.
2. **createRecordFrom (String line)**
Description: Takes a comma-separated string (line) and returns a Class object created from the data in the string.
3. **returnAllRecords ()**
Description: Returns all class records stored in the records list.
4. **contains (String key)**
Description: Checks if a class with the given classID exists in the records list.
5. **getRecord (String key)**
Description: Returns the Class object corresponding to the provided classID.
6. **insertRecord (Class record)**
Description: Adds a new Class to the records list if it doesn't already exist.
7. **deleteRecord (String key)**
Description: Removes the class with the specified classID from the records list and updates the file.
8. **saveToFile ()**
Description: Writes all the class records stored in the records list back to the file.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed	Eng. Ali Hassan
Eng. Ahmed Ashraf	Eng. Seif Eldin Mahmoud
Eng. Miar Mamdouh	Eng. Ahmed AboEleid
Eng. Ahmed Essam	Eng. Menna Tullah Ihab
Eng. Abdelaziz Mohamed	Eng. Mahmoud Ramzy
Eng. Abdelrahman ElSayed	



Fifth Class:

we need to define a class named **MemberClassRegistration**. This class represents a member's registration for a class and handles the registration data. There are three private String variables in the class: **memberID**, **classID**, and **status** (active, cancelled, etc.). There is one private LocalDate variable in the class: **registrationDate**.

No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single constructor that initializes the registration instance attributes.

The class contains five methods:

1. **getMemberID ()**
Description: Returns the id of the member.
2. **getClassID ()**
Description: Returns the id of the class.
3. **getRegistrationDate ()**
Description returns the date of registration.
4. **getSearchKey ()**
Description: Returns a concatenated string (memberID + classID) to uniquely identify the class registration.
5. **lineRepresentation ()**
Description: Returns the class registration information in a comma-separated format (e.g., M1001, C101,2024-10-02, active), used for saving registration information in a file.

The data of the MemberClassRegistration is stored in a file named 'Registration.txt'. Each line represents a data record of one MemberClassRegistration.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. AbdElrahman ElSayed



Sixth Class:

Define a class named **MemberClassRegistrationDatabase** which is responsible for accessing the file of the **MemberClassRegistration** (reading from and writing to), accessing, and manipulating (adding to and removing from) the list of **MemberClassRegistration** objects that hold the data read from the file. There are two private variables in the class:

records (ArrayList< MemberClassRegistration >), filename (String).

No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided. The class has a single constructor, which takes only the file name as parameter Then initializes records ArrayList to prepare the system to handle registration data by reading and writing to the file.

The class contains eight methods:

1. **readFromFile ()**
Description: Reads all class registration records from Registrations.txt, converts them into MemberClassRegistration objects, and stores them in the records list.
2. **createRecordFrom (String line)**
Description: Takes a comma-separated string (line) and returns a MemberClassRegistration object created from the data in the string.
3. **returnAllRecords ()**
Description: Returns all class registration records stored in the records list.
4. **contains (String key)**
Description: Checks if a registration with the given memberID and classID exists in the records list.
5. **getRecord (String key)**
Description: Returns the MemberClassRegistration object for the specified memberID and classID.
6. **insertRecord (MemberClassRegistration record)**
Description: Adds a new MemberClassRegistration object to the records list if it doesn't already exist.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. AbdElrahman ElSayed



7. **deleteRecord (String key)**

Description: Removes the MemberClassRegistration object for the specified memberID and classID from the records list and updates the file.

8. **saveToFile ()**

Description: Writes all class registration records from the records list back to the file.

Seventh Class:

Define a class named **TrainerRole**. This class represents the role of a trainer, responsible for managing members, classes, and class registrations. There are three private variables in the class: **memberDatabase, classDatabase, and registrationDatabase**.

No getters or setters are allowed in this class. Instead, all necessary information will be accessed through the methods provided below. The class has a single construction, which doesn't take any values as parameters and prepares the system to manage trainers by accessing the memberDatabase, classDatabase and registrationDatabase.

The class contains eight methods:

1. **addMember (String memberID, String name, String membershipType, String email, String phoneNumber, String status)**

Description: Adds a new member to the system by inserting a member object into the MemberDatabase. It checks if the member already exists before adding.

2. **getListOfMembers ()**

Description: Returns a list of all members stored in the MemberDatabase.

3. **addClass (String classID, String className, String trainerID, int duration, int maxParticipants)**

Description: Adds a new class to the system by inserting a Class object into the ClassDatabase.

4. **getListOfClasses ()**

Description: Returns a list of all classes stored in the ClassDatabase.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. Abdelrahman ElSayed



5. **registerMemberForClass (String memberID, String classID, LocalDate registrationDate)**

Description: Make sure there are available seats in this class, then register a member by adding a MemberClassRegistration object to the MemberClassRegistrationDatabase. The registration status is set to **"active"**. And decrease the number of available seats by 1. Then returns true if the member registered successfully in the class and false otherwise.

6. **cancelRegistration (String memberID, String classID)**

Description: Cancels a registration and issues a refund if the member cancels within 3 days of the registrationDate. The method searches for registration in the MemberClassRegistrationDatabase, verifies the date, updates the status to **"canceled"** if the cancellation is valid and increases the number of available seats of the class by 1. Then it returns true if the cancellation is valid and false otherwise.

7. **getListOfRegistrations ()**

Description: Returns a list of all class registrations stored in the MemberClassRegistrationDatabase.

8. **logout ()**

Description: Writes all unsaved member, class, and registration data to their respective files before logging out.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. AbdElrahman ElSayed



Requirements:

1. Create an initial UML diagram for the system.
2. Perform refactoring using OOP concepts to improve the initial UML diagram (Refactoring).
3. Write the code based on the refactored UML diagram from Requirement 2, resulting in two UML diagrams and the final code.

Required:

- You are required to solve the above question and deliver a zipped file contains the .java files and two UML Diagrams online through a Google form that will be available for you in next few days.
- The deadline for delivery is **Friday, October 25, 2024, at 11:59 PM**
- A discussion is made with you at your lab time next week on what you have delivered.

Policy:

- You should work in a team of **Two Members** only.
- Delivering a copy will be severely penalized for both parties, so delivering a nothing is so much better than delivering a copy.
- No late submission is allowed.

Dr. Layla Abou-Hadeed

Eng. Ahmed ElSayed

Eng. Ali Hassan

Eng. Ahmed Ashraf

Eng. Seif Eldin Mahmoud

Eng. Miar Mamdouh

Eng. Ahmed AboEleid

Eng. Ahmed Essam

Eng. Menna Tullah Ihab

Eng. Abdelaziz Mohamed

Eng. Mahmoud Ramzy

Eng. AbdElrahman ElSayed